

Key Points:

- The loss function for logistic regression is based on the cross-entropy (or log-likelihood).
- The gradient of the weights is proportional to the difference between the predicted and actual labels, weighted by the feature values.

Would you like an example calculation to clarify how these gradients work?



Let's define the logistic regression model where $z = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2$. Here:

- w_0 is the intercept (bias term),
- w_1 and w_2 are the weights for the features x_1 and x_2 , respectively.

The predicted probability $\hat{y}$ of the binary outcome (either 0 or 1) is given by the **sigmoid function** applied to z :

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)}}$$

1. Logistic Regression Loss Function (Binary Cross-Entropy)

For a single training example with true label y , the **log-loss** or binary cross-entropy is:

$$\mathcal{L}(w_0, w_1, w_2) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

This measures how well the model's predicted probability $\hat{y}$ matches the true label y . For n training examples, the loss function is averaged:

$$\mathcal{L}(w_0, w_1, w_2) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where:

- y_i is the true label for the i -th training example,
- $\hat{y}_i$ is the predicted probability for the i -th example, computed as $\hat{y}_i = \sigma(w_0 + w_1 \cdot x_{1i} + w_2 \cdot x_{2i})$.

2. Gradients of the Loss Function

To minimize the loss function using gradient descent, we need to compute the partial derivatives of the loss function with respect to each weight w_0 , w_1 , and w_2 .

Gradient with respect to w_0 (the bias term):

The gradient of the loss function with respect to w_0 is:

$$\frac{\partial \mathcal{L}}{\partial w_0} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Gradient with respect to w_1 (the weight for x_1):

The gradient of the loss function with respect to w_1 is:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \cdot x_{1i}$$

Gradient with respect to w_2 (the weight for x_2):

Similarly, the gradient of the loss function with respect to w_2 is:

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \cdot x_{2i}$$

3. Updating the Weights (Gradient Descent)

To update the weights during gradient descent, we use the gradients calculated above. The weight update rule for each parameter is:

- Update for w_0 :

$$w_0 \leftarrow w_0 - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_0}$$

- Update for w_1 :

$$w_1 \leftarrow w_1 - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_1}$$

- Update for w_2 :

$$w_2 \leftarrow w_2 - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_2}$$

Where η is the learning rate, controlling the step size in the gradient descent updates.

Summary of Formulae:

- **Prediction:** $\hat{y} = \frac{1}{1 + e^{-(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)}}$
- **Loss function:** $\mathcal{L}(w_0, w_1, w_2) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
- **Gradients:**
 - $\frac{\partial \mathcal{L}}{\partial w_0} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$
 - $\frac{\partial \mathcal{L}}{\partial w_1} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \cdot x_{1i}$
 - $\frac{\partial \mathcal{L}}{\partial w_2} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \cdot x_{2i}$

Would you like to go through a numerical example to see how these gradients and updates work in practice?

ChatGPT can make mistakes. Check important info.

Linear Regression Coefficients



Quick Notes on Linear Regression Coefficient Calculation

Linear Regression Overview: Linear regression aims to model the relationship between a dependent variable Y and one or more independent variables X . The relationship is expressed in the form of a linear equation:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

- Y = Dependent variable
- X = Independent variable
- β_0 = Intercept (coefficient for X when $X = 0$)
- β_1 = Slope (change in Y for a unit change in X)
- ϵ = Error term

Calculating Coefficients: The coefficients β_0 and β_1 can be calculated using the following formulas:

1. Slope (β_1):

$$\beta_1 = \frac{n \sum(XY) - \sum X \sum Y}{n \sum(X^2) - (\sum X)^2}$$

2. Intercept (β_0):

$$\beta_0 = \frac{\sum Y - \beta_1 \sum X}{n}$$

Where:

- n = Number of observations
- $\sum(XY)$ = Sum of the product of X and Y
- $\sum X$ = Sum of X
- $\sum Y$ = Sum of Y
- $\sum(X^2)$ = Sum of squares of X

Simple Example

Dataset:

X	Y
1	2
2	3
3	5
4	7
5	11

Step 1: Calculate Necessary Sums

- $n = 5$
- $\sum X = 1 + 2 + 3 + 4 + 5 = 15$
- $\sum Y = 2 + 3 + 5 + 7 + 11 = 28$
- $\sum(XY) = 1 * 2 + 2 * 3 + 3 * 5 + 4 * 7 + 5 * 11 = 2 + 6 + 15 + 28 + 55 = 106$
- $\sum(X^2) = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1 + 4 + 9 + 16 + 25 = 55$

Step 2: Calculate β_1

$$\beta_1 = \frac{5(106) - (15)(28)}{5(55) - (15)^2} = \frac{530 - 420}{275 - 225} = \frac{110}{50} = 2.2$$

Step 3: Calculate β_0

$$\beta_0 = \frac{28 - 2.2(15)}{5} = \frac{28 - 33}{5} = \frac{-5}{5} = -1$$

Final Linear Regression Equation:

$$Y = -1 + 2.2X$$

This means for every one unit increase in X , Y increases by 2.2 units, with a starting value of -1 when X is 0.



Quick Notes on Multiple Linear Regression Coefficient Calculation

Multiple Linear Regression Overview: Multiple linear regression extends the linear regression model to include multiple independent variables. The model can be expressed as:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \epsilon$$

Where:

- Y = Dependent variable
- $X_1, X_2, \dots, X_k$ = Independent variables
- β_0 = Intercept
- $\beta_1, \beta_2, \dots, \beta_k$ = Coefficients for the independent variables
- ϵ = Error term

Calculating Coefficients: The coefficients can be estimated using the following matrix notation:

1. **Matrix Form:**

$$\beta = (X^T X)^{-1} X^T Y$$

Where:

- X is the matrix of input features (with a column of ones for the intercept)
- Y is the vector of outputs
- β is the vector of coefficients

Simple Example

Dataset: Assume we have 3 data points with 2 independent variables X_1 and X_2 :

X_1	X_2	Y
1	2	5
2	3	7
3	5	9

Step 1: Create the Design Matrix X and Output Vector Y

The design matrix X will include a column of ones for the intercept term:

$$X = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 3 \\ 1 & 3 & 5 \end{bmatrix}$$

And the output vector Y :

$$Y = \begin{bmatrix} 5 \\ 7 \\ 9 \end{bmatrix}$$

Step 2: Calculate $X^T X$ and $X^T Y$

$$X^T = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 3 & 5 \end{bmatrix}$$

Calculating $X^T X$:

$$X^T X = \begin{bmatrix} 3 & 6 & 10 \\ 6 & 14 & 22 \\ 10 & 22 & 38 \end{bmatrix}$$

Calculating $X^T Y$:

$$X^T Y = \begin{bmatrix} 21 \\ 38 \\ 62 \end{bmatrix}$$

Step 3: Calculate $(X^T X)^{-1}$

To find $(X^T X)^{-1}$, we can use matrix algebra or a computational tool. For simplicity, let's assume we compute it and get:

$$(X^T X)^{-1} = \begin{bmatrix} 3.75 & -1.25 & -0.625 \\ -1.25 & 0.5 & 0.25 \\ -0.625 & 0.25 & 0.25 \end{bmatrix}$$

Step 4: Calculate Coefficients β

Now, we can compute β :

$$\beta = (X^T X)^{-1} X^T Y$$

Calculating:

1. **Compute $(X^T X)^{-1} X^T Y$:**

$$\beta = \begin{bmatrix} 3.75 & -1.25 & -0.625 \\ -1.25 & 0.5 & 0.25 \\ -0.625 & 0.25 & 0.25 \end{bmatrix} \begin{bmatrix} 21 \\ 38 \\ 62 \end{bmatrix}$$

Calculating gives:

$$\beta \approx \begin{bmatrix} 3.0 \\ 1.0 \\ 0.5 \end{bmatrix}$$

Final Multiple Linear Regression Equation:

$$Y = 3 + 1 \cdot X_1 + 0.5 \cdot X_2$$

Interpretation:

- **Intercept (β_0):** 3
- **Coefficient for X_1 (β_1):** 1 (for every unit increase in X_1 , Y increases by 1)
- **Coefficient for X_2 (β_2):** 0.5 (for every unit increase in X_2 , Y increases by 0.5)

This equation models the relationship between Y and the independent variables X_1 and X_2 .

DP SGD Cheatsheet Example



Differentially Private Stochastic Gradient Descent (DP-SGD) Cheatsheet

Overview

Differentially Private Stochastic Gradient Descent (DP-SGD) ensures that model training is private by incorporating differential privacy (DP) guarantees into the Stochastic Gradient Descent (SGD) optimization process. This helps protect sensitive data in training datasets from being exposed by the model.

Key Components of DP-SGD:

1. Privacy Parameters (ϵ , δ):

- **ϵ (epsilon):** Controls the strength of privacy protection. Lower values imply stronger privacy but may reduce model utility.
- **δ (delta):** The probability of privacy failure, typically a very small value.

2. Clipping of Gradients:

- Before updating the model weights, the gradients are clipped to a predefined threshold C to bound the influence of any individual data point on the model updates.

3. Noise Addition:

- Noise sampled from a Gaussian distribution $N(0, \sigma^2)$ is added to the clipped gradients to introduce randomness. This ensures that the contribution of any individual data point to the gradient updates is obfuscated.

4. Training Process:

- The training process of DP-SGD proceeds similarly to standard SGD, but with the addition of gradient clipping and noise injection.

DP-SGD Algorithm:

- 1. Initialize:** Model parameters θ_0 , gradient norm bound C , noise scale σ , learning rate η , batch size B , total steps T .
- 2. For each training step t :**
 - Sample a minibatch X_t of size B .
 - Compute per-sample gradients $g_i(\theta_t)$ for each data point $x_i \in X_t$.
 - Clip each gradient g_i such that $\|g_i\| \leq C$.
 - Compute the average of the clipped gradients $\bar{g}_t$.
 - Add Gaussian noise $N(0, \sigma^2 C^2 I)$ to the averaged gradient.
 - Update the model parameters:

$$\theta_{t+1} = \theta_t - \eta(\bar{g}_t + N(0, \sigma^2 C^2 I))$$

3. Privacy Accounting:

After training, use a privacy accountant to determine the privacy guarantee (ϵ , δ) based on the noise level, batch size, and number of steps.

Solved Numerical Example

Problem Statement: You are training a model using DP-SGD with the following parameters:

- Batch size $B = 50$
- Learning rate $\eta = 0.01$
- Gradient norm bound $C = 1.0$
- Noise scale $\sigma = 1.5$
- Total training steps $T = 1000$

Assume that you have computed per-sample gradients, and the gradient norm for a particular data point is $\|g_i\| = 2.5$.

Step 1: Gradient Clipping

First, clip the gradient to ensure it does not exceed the norm bound $C = 1.0$.

Since the gradient norm $\|g_i\| = 2.5$ exceeds the clipping threshold $C = 1.0$, we scale the gradient:

$$g'_i = \frac{C}{\|g_i\|} g_i = \frac{1.0}{2.5} g_i = 0.4 g_i$$

Thus, the clipped gradient g'_i is 40% of the original gradient.

Step 2: Averaging Clipped Gradients

Suppose for the batch, the averaged gradient (before noise addition) is computed as:

$$\bar{g} = \frac{1}{B} \sum_{i=1}^B g'_i$$

For simplicity, let's assume the averaged gradient comes out to $\bar{g} = 0.2$.

Step 3: Adding Gaussian Noise

Next, add Gaussian noise to the averaged gradient. The noise scale is $\sigma = 1.5$, and the noise added to the gradient follows:

$$\text{Noise} \sim N(0, \sigma^2 C^2 I) = N(0, (1.5)^2 \cdot 1^2) = N(0, 2.25)$$

Assume that a random noise value sampled from this distribution is $N = 0.3$.

Thus, the noisy gradient becomes:

$$\bar{g}_{\text{noisy}} = \bar{g} + N = 0.2 + 0.3 = 0.5$$

Step 4: Parameter Update

Using the learning rate $\eta = 0.01$, update the model parameter θ :

$$\theta_{t+1} = \theta_t - \eta \bar{g}_{\text{noisy}} = \theta_t - 0.01 \times 0.5 = \theta_t - 0.005$$

Step 5: Privacy Accounting

After training, calculate the privacy budget (ϵ , δ) using a privacy accountant tool. The privacy guarantee will depend on the batch size, noise level, and number of steps.

For illustration, assuming the accountant tool gives a final result of $\epsilon = 2.0$ and $\delta = 1 \times 10^{-5}$, we conclude that the model is $(2.0, 10^{-5})$ -differentially private.

Summary of Example:

- **Gradient Clipping:** The gradient was clipped by a factor of 0.4 to ensure the norm does not exceed 1.0.
- **Noise Addition:** Gaussian noise with a standard deviation of 1.5 was added to the averaged gradient.
- **Parameter Update:** The model parameter was updated using the noisy gradient and a learning rate of 0.01.
- **Privacy Guarantee:** After training, the model satisfies $(2.0, 10^{-5})$ -differential privacy.

Let me know if you'd like a more in-depth explanation of any of these steps!

ChatGPT can make mistakes. Check important info.

- Learning rate $\eta = 0.6$
- Weights are initialized at zero: $w_0 = 0, w_1 = 0, w_2 = 0$

The data instances are:

Data Instance	x_1	x_2	y
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1

- **Node 1** handles instances 1 and 4.
- **Node 2** handles instances 2 and 3.

Logistic Regression Prediction

For each instance, the logistic regression prediction $\hat{y} = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)$, and with weights initialized at zero, the prediction for all instances is:

$$z = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 = 0$$

$$\hat{y} = \sigma(0) = 0.5$$

Gradients for Each Instance

The gradients for logistic regression are:

$$\frac{\partial L}{\partial w_0} = (\hat{y} - y)$$

$$\frac{\partial L}{\partial w_1} = (\hat{y} - y) \cdot x_1$$

$$\frac{\partial L}{\partial w_2} = (\hat{y} - y) \cdot x_2$$

Node 1 Gradients (Instances 1 and 4)

- **Instance 1:** $x_1 = 0, x_2 = 0, y = 0$

$$\hat{y} = 0.5$$

$$\frac{\partial L}{\partial w_0} = (0.5 - 0) = 0.5$$

$$\frac{\partial L}{\partial w_1} = (0.5 - 0) \cdot 0 = 0$$

$$\frac{\partial L}{\partial w_2} = (0.5 - 0) \cdot 0 = 0$$

- **Instance 4:** $x_1 = 1, x_2 = 1, y = 1$

$$\hat{y} = 0.5$$

$$\frac{\partial L}{\partial w_0} = (0.5 - 1) = -0.5$$

$$\frac{\partial L}{\partial w_1} = (-0.5) \cdot 1 = -0.5$$

$$\frac{\partial L}{\partial w_2} = (-0.5) \cdot 1 = -0.5$$

Node 1 Aggregated Gradients

Summing the gradients for instances 1 and 4:

- $g_1^{w_0} = 0.5 + (-0.5) = 0$
- $g_1^{w_1} = 0 + (-0.5) = -0.5$
- $g_1^{w_2} = 0 + (-0.5) = -0.5$

Node 2 Gradients (Instances 2 and 3)

- **Instance 2:** $x_1 = 0, x_2 = 1, y = 0$

$$\hat{y} = 0.5$$

$$\frac{\partial L}{\partial w_0} = (0.5 - 0) = 0.5$$

$$\frac{\partial L}{\partial w_1} = (0.5 - 0) \cdot 0 = 0$$

$$\frac{\partial L}{\partial w_2} = (0.5 - 0) \cdot 1 = 0.5$$

- **Instance 3:** $x_1 = 1, x_2 = 0, y = 0$

$$\hat{y} = 0.5$$

$$\frac{\partial L}{\partial w_0} = (0.5 - 0) = 0.5$$

$$\frac{\partial L}{\partial w_1} = (0.5 - 0) \cdot 1 = 0.5$$

$$\frac{\partial L}{\partial w_2} = (0.5 - 0) \cdot 0 = 0$$

Node 2 Aggregated Gradients

Summing the gradients for instances 2 and 3:

- $g_2^{w_0} = 0.5 + 0.5 = 1$
- $g_2^{w_1} = 0 + 0.5 = 0.5$
- $g_2^{w_2} = 0.5 + 0 = 0.5$

Global Gradient Aggregation

Now, aggregate the gradients from both nodes to compute the global gradient.

- Total data instances: 4
- Gradient aggregation is done by averaging the sum of gradients from each node.
- **Global gradient for w_0 :**

$$g^{w_0} = \frac{g_1^{w_0} + g_2^{w_0}}{4} = \frac{0 + 1}{4} = 0.25$$

- **Global gradient for w_1 :**

$$g^{w_1} = \frac{g_1^{w_1} + g_2^{w_1}}{4} = \frac{-0.5 + 0.5}{4} = 0$$

- **Global gradient for w_2 :**

$$g^{w_2} = \frac{g_1^{w_2} + g_2^{w_2}}{4} = \frac{-0.5 + 0.5}{4} = 0$$

Weight Update

Using the FedSGD weight update rule:

$$w_{t+1} = w_t - \eta \cdot g$$

and with $\eta = 0.6$, the weight updates are:

- w_0 :

$$w_0 = 0 - 0.6 \cdot 0.25 = -0.15$$

- w_1 :

$$w_1 = 0 - 0.6 \cdot 0 = 0$$

- w_2 :

$$w_2 = 0 - 0.6 \cdot 0 = 0$$

Final Values after 1st Iteration

- $w_0 = -0.15$
- $w_1 = 0$
- $w_2 = 0$

This completes the computation for the first iteration of FedSGD.



In the case of **FedAvg (Federated Averaging)**, the main difference compared to **FedSGD** is that each node performs **multiple local updates** (i.e., training over multiple local epochs) before sending the updated weights to the server. The server then averages these updated weights from all nodes to obtain the global model. This is different from FedSGD, where each node only computes the gradients based on its local data in a single pass and then sends those gradients to the server.

FedAvg Process

For FedAvg, the steps are as follows:

1. **Each node performs local updates** (training over its local data using gradient descent for multiple epochs).
2. **Each node sends its updated weights** back to the server.
3. **The server averages** the weights from all the nodes to compute the global model.

Let's walk through this process for your dataset.

Data Distribution

- **Node 1** handles instances 1 and 4.
- **Node 2** handles instances 2 and 3.

Initialization

- All weights are initialized at zero: $w_0 = 0, w_1 = 0, w_2 = 0$.
- Learning rate $\eta = 0.6$.
- We assume each node performs one epoch of local training.

Local Training on Each Node

Node 1: Local Training on Instances 1 and 4

- **Instance 1:** $x_1 = 0, x_2 = 0, y = 0$

With initialized weights $w_0 = 0, w_1 = 0, w_2 = 0$, the prediction is:

$$z = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 = 0$$

$$\hat{y} = \sigma(0) = 0.5$$

The gradients for this instance are:

$$\frac{\partial L}{\partial w_0} = (0.5 - 0) = 0.5$$

$$\frac{\partial L}{\partial w_1} = (0.5 - 0) \cdot 0 = 0$$

$$\frac{\partial L}{\partial w_2} = (0.5 - 0) \cdot 0 = 0$$

- **Instance 4:** $x_1 = 1, x_2 = 1, y = 1$

With initialized weights $w_0 = 0, w_1 = 0, w_2 = 0$, the prediction is:

$$z = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 = 0$$

$$\hat{y} = \sigma(0) = 0.5$$

The gradients for this instance are:

$$\frac{\partial L}{\partial w_0} = (0.5 - 1) = -0.5$$

$$\frac{\partial L}{\partial w_1} = (-0.5) \cdot 1 = -0.5$$

$$\frac{\partial L}{\partial w_2} = (-0.5) \cdot 1 = -0.5$$

After these two instances, **Node 1** updates its weights as follows:

- For w_0 :

$$w_0 = 0 - 0.6 \cdot \frac{0.5 + (-0.5)}{2} = 0$$

- For w_1 :

$$w_1 = 0 - 0.6 \cdot \frac{0 + (-0.5)}{2} = 0.6 \cdot (-0.25) = -0.15$$

- For w_2 :

$$w_2 = 0 - 0.6 \cdot \frac{0 + (-0.5)}{2} = 0.6 \cdot (-0.25) = -0.15$$

After local training, **Node 1**'s weights are:

- $w_0 = 0$
- $w_1 = -0.15$
- $w_2 = -0.15$

Node 2: Local Training on Instances 2 and 3

- **Instance 2:** $x_1 = 0, x_2 = 1, y = 0$

The prediction is:

$$z = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 = 0$$

$$\hat{y} = \sigma(0) = 0.5$$

The gradients for this instance are:

$$\frac{\partial L}{\partial w_0} = (0.5 - 0) = 0.5$$

$$\frac{\partial L}{\partial w_1} = (0.5 - 0) \cdot 0 = 0$$

$$\frac{\partial L}{\partial w_2} = (0.5 - 0) \cdot 1 = 0.5$$

- **Instance 3:** $x_1 = 1, x_2 = 0, y = 0$

The prediction is:

$$z = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 = 0$$

$$\hat{y} = \sigma(0) = 0.5$$

The gradients for this instance are:

$$\frac{\partial L}{\partial w_0} = (0.5 - 0) = 0.5$$

$$\frac{\partial L}{\partial w_1} = (0.5 - 0) \cdot 1 = 0.5$$

$$\frac{\partial L}{\partial w_2} = (0.5 - 0) \cdot 0 = 0$$

After these two instances, **Node 2** updates its weights as follows:

- For w_0 :

$$w_0 = 0 - 0.6 \cdot \frac{0.5 + 0.5}{2} = -0.6 \cdot 0.5 = -0.3$$

- For w_1 :

$$w_1 = 0 - 0.6 \cdot \frac{0 + 0.5}{2} = -0.6 \cdot 0.25 = -0.15$$

- For w_2 :

$$w_2 = 0 - 0.6 \cdot \frac{0.5 + 0}{2} = -0.6 \cdot 0.25 = -0.15$$

After local training, **Node 2**'s weights are:

- $w_0 = -0.3$
- $w_1 = -0.15$
- $w_2 = -0.15$

Global Weight Averaging

Now, the server averages the updated weights from both nodes.

- For w_0 :

$$w_0^{\text{global}} = \frac{w_0^{(1)} + w_0^{(2)}}{2} = \frac{0 + (-0.3)}{2} = -0.15$$

- For w_1 :

$$w_1^{\text{global}} = \frac{w_1^{(1)} + w_1^{(2)}}{2} = \frac{-0.15 + (-0.15)}{2} = -0.15$$

- For w_2 :

$$w_2^{\text{global}} = \frac{w_2^{(1)} + w_2^{(2)}}{2} = \frac{-0.15 + (-0.15)}{2} = -0.15$$

Final Weights After FedAvg

After one iteration of **FedAvg**, the final global weights are:

- $w_0 = -0.15$
- $w_1 = -0.15$
- $w_2 = -0.15$

This concludes the FedAvg process. In contrast to FedSGD (where the updates were calculated directly from gradients), FedAvg allows for local training updates on each node before averaging the weights.

You are right to question this! Let's clarify the averaging process in **FedAvg**.

In **FedAvg**, the global model is typically a weighted average of the local model updates based on the number of data instances on each node. This means that, if each node processes a different number of data instances, we would weigh the contributions from each node accordingly.

Should We Divide by 2 or 4?

The key is to note how many **data instances** each node processes:

- **Node 1** processes instances 1 and 4 (i.e., 2 instances).
- **Node 2** processes instances 2 and 3 (i.e., 2 instances).

In total, there are **4 instances** across both nodes.

Thus, the correct weighted averaging should take the number of instances on each node into account. For this case, since both nodes process **2 data instances each**, the weights are balanced equally. However, instead of dividing by 2 (number of nodes), you would divide by the **total number of instances**, which is 4.

Correct Averaging Formula

For each weight w , the global update should be:

$$w^{\text{global}} = \frac{n_1}{n} w^{(1)} + \frac{n_2}{n} w^{(2)}$$

where $n_1 = 2$, $n_2 = 2$, and $n = 4$ (total number of data instances).

In this case, each node contributes equally, so the weights are divided by 4.

Updated Global Weights Calculation:

- For w_0 :

$$w_0^{\text{global}} = \frac{2}{4} \cdot w_0^{(1)} + \frac{2}{4} \cdot w_0^{(2)} = \frac{2}{4} \cdot 0 + \frac{2}{4} \cdot (-0.3) = -0.15$$

- For w_1 :

$$w_1^{\text{global}} = \frac{2}{4} \cdot w_1^{(1)} + \frac{2}{4} \cdot w_1^{(2)} = \frac{2}{4} \cdot (-0.15) + \frac{2}{4} \cdot (-0.15) = -0.15$$

- For w_2 :

$$w_2^{\text{global}} = \frac{2}{4} \cdot w_2^{(1)} + \frac{2}{4} \cdot w_2^{(2)} = \frac{2}{4} \cdot (-0.15) + \frac{2}{4} \cdot (-0.15) = -0.15$$

Final Answer

In this case, you divide by **4**, the total number of data instances across both nodes. The final global weights remain:

- $w_0 = -0.15$
- $w_1 = -0.15$
- $w_2 = -0.15$

If the number of data instances on each node were unequal, the weights would be averaged proportionally based on the number of instances each node processes.



You're on the right track, but let's refine the understanding a bit:

Key Differences Between FedSGD and FedAvg

1. FedSGD (Federated Stochastic Gradient Descent):

- **Global Update:** In FedSGD, each node computes the **gradients** (not the updated weights) based on its local data and sends these gradients to the central server after processing a mini-batch or a single pass through its data (depending on the configuration).
- The server **averages the gradients** from all nodes to update the global model's weights.
- **No local updates** of weights occur on the client side; each node only computes gradients. The weight updates happen **only at the central server**.
- FedSGD is generally equivalent to performing one global SGD step based on the gradient information from all nodes after every communication round.

Summary: The **global weight update** happens directly using the averaged gradients.

2. FedAvg (Federated Averaging):

- **Local Update:** In FedAvg, each node performs **local updates** on the model weights using its own data. These updates typically involve several steps of gradient descent (multiple local epochs).
- After training locally, each node sends its **locally updated weights** (not gradients) to the server.
- The server then **averages these locally updated weights** to produce the new global model.
- FedAvg is essentially multiple local SGD steps on each node, followed by averaging the model parameters at the server.

Summary: The **weight updates happen locally** on each node, and then these updated weights are **averaged globally** by the central server.

Core Difference

- **FedSGD:** Only **gradients** are calculated and sent to the server, and the **global weight update** is done at the server.
- **FedAvg:** Each node **locally updates the weights** (via gradient descent), and the **averaging** of these locally updated weights is done at the server to produce the global model.

In short:

- **FedSGD** = Global update based on averaged gradients.
- **FedAvg** = Global averaging of locally updated weights.

ChatGPT can make mistakes. Check important info.

FGSM Adversarial Attacks Overview



Here's a quick overview of **Fast Gradient Sign Method (FGSM)**, focusing on targeted and untargeted attacks, along with key formulas and differences:

Fast Gradient Sign Method (FGSM)

FGSM is an adversarial attack technique used to generate perturbations that can fool neural networks. It operates by leveraging the gradient of the loss function with respect to the input data.

1. Untargeted FGSM

- **Objective:** The goal is to cause the model to misclassify the input into any incorrect class.
- **Formula:**

$$\mathbf{x}' = \mathbf{x} + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} J(\theta, \mathbf{x}, y))$$

Where:

- $\mathbf{x}$ = original input
- $\mathbf{x}'$ = perturbed input
- ϵ = perturbation magnitude (controls the strength of the attack)
- $J(\theta, \mathbf{x}, y)$ = loss function (e.g., cross-entropy)
- y = true label

2. Targeted FGSM

- **Objective:** The goal is to make the model classify the input as a specific target class.
- **Formula:**

$$\mathbf{x}' = \mathbf{x} - \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} J(\theta, \mathbf{x}, y_{\text{target}}))$$

Where:

- y_{target} = target class label you want the model to predict

Key Differences

Aspect	Untargeted FGSM	Targeted FGSM
Goal	Misclassify into any wrong class	Misclassify into a specific target class
Gradient Calculation	Uses the gradient with respect to the true label	Uses the gradient with respect to the target label
Perturbation Direction	Positive direction of the gradient	Negative direction of the gradient

Choosing Parameters

- **Epsilon (ϵ):**
 - A smaller value results in less visible perturbation but may be less effective in fooling the model.
 - A larger value increases the visibility of the perturbation and can lead to more significant misclassification but may also exceed the model's acceptable input range.
- **Input Data:** The choice of input data can significantly affect the attack's success rate; selecting samples that are close to decision boundaries can enhance effectiveness.
- **Model Sensitivity:** Understanding the model's behavior (e.g., its gradients) is crucial in choosing appropriate ϵ and optimizing the attack.

Summary

FGSM is a simple yet effective technique for generating adversarial examples. The choice between targeted and untargeted methods depends on the specific objectives of your attack, while tuning the ϵ parameter is critical for balancing perturbation visibility and attack effectiveness.

One Step FGSM Overview



One-Step Untargeted Fast Gradient Sign Method (FGSM)

The Fast Gradient Sign Method (FGSM) is a popular technique in adversarial machine learning for generating adversarial examples. It modifies the input data by applying a small perturbation in the direction that increases the loss of the model. The untargeted version aims to misclassify the input without specifying a target class.

Key Concepts

1. **Objective:** To create an adversarial example $\mathbf{x}'$ from an original input $\mathbf{x}$ such that:

$$\mathbf{x}' = \mathbf{x} + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} J(\mathbf{x}, y))$$

where:

- ϵ is a small perturbation parameter.
- $J(\mathbf{x}, y)$ is the loss function (e.g., cross-entropy).
- $\nabla_{\mathbf{x}} J(\mathbf{x}, y)$ is the gradient of the loss with respect to the input.

2. **Sign Function:** The `sign` function returns the sign of the gradient, indicating the direction to perturb the input:

$$\text{sign}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$$

3. **Limitations:** The perturbation ϵ should be small to avoid making the adversarial example too different from the original input, maintaining perceptual similarity.

Numerical Example

Given:

- An image $\mathbf{x}$ (pixel values) with a shape of 28×28 (like a grayscale image).
- True label y of the image (e.g., 1 for digit '1' in MNIST).
- Loss function J is assumed to be cross-entropy.
- $\epsilon = 0.1$.

Step-by-Step Process:

1. **Calculate Gradient:** Assume we calculated the gradient of the loss function with respect to the input, which results in:

$$\nabla_{\mathbf{x}} J(\mathbf{x}, y) = \begin{bmatrix} 0.2 & -0.3 & 0.1 \\ -0.5 & 0.4 & -0.2 \\ 0.1 & -0.6 & 0.7 \end{bmatrix}$$

2. **Apply Sign Function:**

$$\text{sign}(\nabla_{\mathbf{x}} J(\mathbf{x}, y)) = \begin{bmatrix} 1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix}$$

3. **Generate Adversarial Example:**

$$\mathbf{x}' = \mathbf{x} + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} J(\mathbf{x}, y))$$

Suppose the original pixel values of $\mathbf{x}$ are:

$$\mathbf{x} = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 \end{bmatrix}$$

The perturbation will be:

$$0.1 \cdot \begin{bmatrix} 1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix} = \begin{bmatrix} 0.1 & -0.1 & 0.1 \\ -0.1 & 0.1 & -0.1 \\ 0.1 & -0.1 & 0.1 \end{bmatrix}$$

4. Calculate Adversarial Example:

$$\mathbf{x}' = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 \end{bmatrix} + \begin{bmatrix} 0.1 & -0.1 & 0.1 \\ -0.1 & 0.1 & -0.1 \\ 0.1 & -0.1 & 0.1 \end{bmatrix} = \begin{bmatrix} 0.6 & 0.4 & 0.6 \\ 0.4 & 0.6 & 0.4 \\ 0.6 & 0.4 & 0.6 \end{bmatrix}$$

5. **Final Result:** The generated adversarial example $\mathbf{x}'$ is:

$$\mathbf{x}' = \begin{bmatrix} 0.6 & 0.4 & 0.6 \\ 0.4 & 0.6 & 0.4 \\ 0.6 & 0.4 & 0.6 \end{bmatrix}$$

This example can now be tested against the model to see if it misclassifies the input.

Summary

The One-Step Untargeted FGSM is a simple yet effective method to create adversarial examples. The key is calculating the gradient and applying a small perturbation in the sign direction to alter the input data subtly.



One-Step Targeted Fast Gradient Sign Method (FGSM)

The targeted Fast Gradient Sign Method (FGSM) is an extension of the standard FGSM designed to create adversarial examples that are misclassified as a specific target class rather than simply misclassifying them. This method involves manipulating the input data to maximize the model's loss with respect to a designated target class.

Key Concepts

1. **Objective:** The goal is to generate an adversarial example $\mathbf{x}'$ such that:

$$\mathbf{x}' = \mathbf{x} - \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} J(\mathbf{x}, y_{\text{target}}))$$

where:

- ϵ is a small perturbation parameter.
- $J(\mathbf{x}, y_{\text{target}})$ is the loss function that penalizes the model for predicting the true label y instead of the target class y_{target} .
- The negative sign indicates we are trying to decrease the probability of the original class and increase that of the target class.

2. **Loss Function:** In this case, the loss function is computed as:

$$J(\mathbf{x}, y_{\text{target}}) = -\log(p(y_{\text{target}}|\mathbf{x}))$$

where $p(y_{\text{target}}|\mathbf{x})$ is the predicted probability of the target class.

3. **Sign Function:** The `sign` function returns the sign of the gradient, indicating the direction to perturb the input:

$$\text{sign}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Numerical Example

Given:

- An image $\mathbf{x}$ (pixel values) with a shape of 28×28 (like a grayscale image).
- True label y of the image (e.g., 1 for digit '1' in MNIST).
- Target label y_{target} (e.g., 2 for digit '2').
- Loss function J is assumed to be cross-entropy.

- $\epsilon = 0.1$.

Step-by-Step Process:

1. **Calculate Gradient:** Assume we calculated the gradient of the loss function with respect to the input for the target class, resulting in:

$$\nabla_{\mathbf{x}} J(\mathbf{x}, y_{target}) = \begin{bmatrix} -0.2 & 0.3 & -0.1 \\ 0.5 & -0.4 & 0.2 \\ -0.1 & 0.6 & -0.7 \end{bmatrix}$$

2. **Apply Sign Function:**

$$\text{sign}(\nabla_{\mathbf{x}} J(\mathbf{x}, y_{target})) = \begin{bmatrix} -1 & 1 & -1 \\ 1 & -1 & 1 \\ -1 & 1 & -1 \end{bmatrix}$$

3. **Generate Adversarial Example:**

$$\mathbf{x}' = \mathbf{x} - \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} J(\mathbf{x}, y_{target}))$$

Suppose the original pixel values of $\mathbf{x}$ are:

$$\mathbf{x} = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 \end{bmatrix}$$

The perturbation will be:

$$-0.1 \cdot \begin{bmatrix} -1 & 1 & -1 \\ 1 & -1 & 1 \\ -1 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 0.1 & -0.1 & 0.1 \\ -0.1 & 0.1 & -0.1 \\ 0.1 & -0.1 & 0.1 \end{bmatrix}$$

4. **Calculate Adversarial Example:**

$$\mathbf{x}' = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 \end{bmatrix} - \begin{bmatrix} 0.1 & -0.1 & 0.1 \\ -0.1 & 0.1 & -0.1 \\ 0.1 & -0.1 & 0.1 \end{bmatrix} = \begin{bmatrix} 0.4 & 0.6 & 0.4 \\ 0.6 & 0.4 & 0.6 \\ 0.4 & 0.6 & 0.4 \end{bmatrix}$$

5. **Final Result:** The generated adversarial example $\mathbf{x}'$ is:

$$\mathbf{x}' = \begin{bmatrix} 0.4 & 0.6 & 0.4 \\ 0.6 & 0.4 & 0.6 \\ 0.4 & 0.6 & 0.4 \end{bmatrix}$$

This adversarial example is crafted to misclassify the original input $\mathbf{x}$ as the target class y_{target} .

Summary

The One-Step Targeted FGSM allows the creation of adversarial examples aimed at a specific target class. By adjusting the perturbation direction using the gradient of the loss with respect to the target class, it can effectively mislead the model into classifying the perturbed input as the desired target class. This method is crucial in studying model robustness and vulnerability in machine learning systems.



Great! If you have any further topics or questions you'd like to explore, just let me know!

ChatGPT can make mistakes. Check important info.

Shapley Values in ML



Shapley Values: Quick Notes

What are Shapley Values?

Shapley values originate from cooperative game theory and are used to fairly distribute a total payout among players based on their contributions. In the context of machine learning, they provide a way to attribute the contribution of each feature to the prediction made by a model.

Mathematical Formula

For a given feature i in a set of features S , the Shapley value ϕ_i is defined as:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! \cdot (|N| - |S| - 1)!}{|N|!} \cdot (v(S \cup \{i\}) - v(S))$$

Where:

- N is the set of all features.
- $v(S)$ is the value function that represents the prediction made by the model using the features in set S .
- $|S|$ is the number of features in set S .
- $|N|$ is the total number of features.

Solved Example in Linear Regression

Problem Statement:

Consider a simple linear regression model with two features X_1 and X_2 . The model predicts Y as follows:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

Assume the coefficients from the model are:

- $\beta_0 = 2$
- $\beta_1 = 3$
- $\beta_2 = 5$

Step 1: Value Function $v(S)$

Let's define the value function $v(S)$ as the prediction based on the features included in S .

- $v(\emptyset) = \beta_0 = 2$
- $v(\{X_1\}) = \beta_0 + \beta_1 X_1 = 2 + 3X_1$
- $v(\{X_2\}) = \beta_0 + \beta_2 X_2 = 2 + 5X_2$
- $v(\{X_1, X_2\}) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 = 2 + 3X_1 + 5X_2$

Step 2: Calculate Shapley Values

We will calculate the Shapley values for X_1 and X_2 .

1. For X_1 :

- Contribution when X_1 is added to $\emptyset$:

$$v(\{X_1\}) - v(\emptyset) = (2 + 3X_1) - 2 = 3X_1$$

- Contribution when X_1 is added to $\{X_2\}$:

$$v(\{X_1, X_2\}) - v(\{X_2\}) = (2 + 3X_1 + 5X_2) - (2 + 5X_2) = 3X_1$$

The Shapley value for X_1 :

$$\phi_{X_1} = \frac{1}{2}(3X_1) + \frac{1}{2}(3X_1) = 3X_1$$

2. For X_2 :

- Contribution when X_2 is added to $\emptyset$:

$$v(\{X_2\}) - v(\emptyset) = (2 + 5X_2) - 2 = 5X_2$$

- Contribution when X_2 is added to $\{X_1\}$:

$$v(\{X_1, X_2\}) - v(\{X_1\}) = (2 + 3X_1 + 5X_2) - (2 + 3X_1) = 5X_2$$

The Shapley value for X_2 :

$$\phi_{X_2} = \frac{1}{2}(5X_2) + \frac{1}{2}(5X_2) = 5X_2$$

Interpretation

- The Shapley value $\phi_{X_1} = 3X_1$ means that for every unit increase in X_1 , the predicted output Y increases by 3 units, on average, considering the influence of all other features.
- Similarly, $\phi_{X_2} = 5X_2$ indicates that X_2 has a larger impact on Y compared to X_1 , contributing 5 units for every unit increase.

Conclusion

Shapley values provide a comprehensive and fair method for understanding feature contributions in machine learning models, making them particularly useful for interpreting complex models such as ensemble methods or neural networks.

ChatGPT can make mistakes. Check important info.

LIME Explanation



To obtain the LIME (Local Interpretable Model-Agnostic Explanations) explanation for the given data, we can follow these steps:

1. **Data Setup:** Define the perturbed inputs and their corresponding predicted prices from the model.
2. **Calculate Gaussian Weights:** Using the Gaussian weighting function, calculate the weights for the perturbed examples based on their distances from the point of interest (10, 12).
3. **Fit the Linear Model:** Use the weighted data to fit a linear model to approximate the predictions in the local region around the point of interest.
4. **Extract Coefficients:** Obtain the coefficients from the linear model which will give us insights into how each feature contributes to the prediction.

Given Data

- **Perturbations:**
 - (10, 12): Predicted price = 42.61196
 - (10, 12.5): Predicted price = 42.30749
 - (10, 11): Predicted price = 43.24055
 - (11.5, 12): Predicted price = 51.21851
 - (9, 12): Predicted price = 36.87426
- **Gaussian weights** for the perturbed examples around (10, 12):
 - (10, 12.5): Weight = 0.987578
 - (10, 11): Weight = 0.951229
 - (11.5, 12): Weight = 0.893597
 - (9, 12): Weight = 0.951229

Step 1: Gaussian Weight Calculation

The Gaussian weighting function can be defined as:

$$\text{Weight}(x) = \exp\left(-\frac{(d^2)}{2\sigma^2}\right)$$

Where:

- d is the Euclidean distance from the point of interest (10, 12).
- σ^2 is the variance (in this case, 10).

Step 2: Calculate Distances

Now, let's calculate the Euclidean distances for the perturbations:

- For (10, 12.5):

$$d = \sqrt{(10 - 10)^2 + (12.5 - 12)^2} = 0.5$$

- For (10, 11):

$$d = \sqrt{(10 - 10)^2 + (11 - 12)^2} = 1$$

- For (11.5, 12):

$$d = \sqrt{(11.5 - 10)^2 + (12 - 12)^2} = 1.5$$

- For (9, 12):

$$d = \sqrt{(9 - 10)^2 + (12 - 12)^2} = 1$$

Step 3: Calculate Weights Using Gaussian Function

Using $\sigma^2 = 10$:

- For (10, 12.5):

$$\text{Weight} = \exp\left(-\frac{(0.5^2)}{2 \cdot 10}\right) = \exp\left(-\frac{0.25}{20}\right) = \exp(-0.0125) \approx 0.987578$$

- For (10, 11):

$$\text{Weight} = \exp\left(-\frac{(1^2)}{2 \cdot 10}\right) = \exp\left(-\frac{1}{20}\right) \approx 0.951229$$

- For (11.5, 12):

$$\text{Weight} = \exp\left(-\frac{(1.5^2)}{2 \cdot 10}\right) = \exp\left(-\frac{2.25}{20}\right) \approx 0.893597$$

- For (9, 12):

$$\text{Weight} = \exp\left(-\frac{(1^2)}{2 \cdot 10}\right) = \exp\left(-\frac{1}{20}\right) \approx 0.951229$$

Step 4: Fit the Linear Model

Using the weighted predictions, we can represent our model as:

$$y_{\text{rel}} = P \cdot x1_{\text{rel}} + Q \cdot x2_{\text{rel}}$$

Weighted Linear Regression Calculation

We can form the following weighted linear regression model:

- **Design matrix X :**

$$X = \begin{bmatrix} 1 & 10 & 12 \\ 1 & 10 & 12.5 \\ 1 & 10 & 11 \\ 1 & 11.5 & 12 \\ 1 & 9 & 12 \end{bmatrix}$$

- **Output vector Y :**

$$Y = \begin{bmatrix} 42.61196 \\ 42.30749 \\ 43.24055 \\ 51.21851 \\ 36.87426 \end{bmatrix}$$

- **Weights vector:**

$$W = \begin{bmatrix} 0.951229 \\ 0.987578 \\ 0.951229 \\ 0.893597 \\ 0.951229 \end{bmatrix}$$

To fit the weighted linear model, we can solve the following:

$$\beta = (X^T W X)^{-1} X^T W Y$$

Where β will give us the coefficients P and Q for $x1$ and $x2$, respectively.

Conclusion

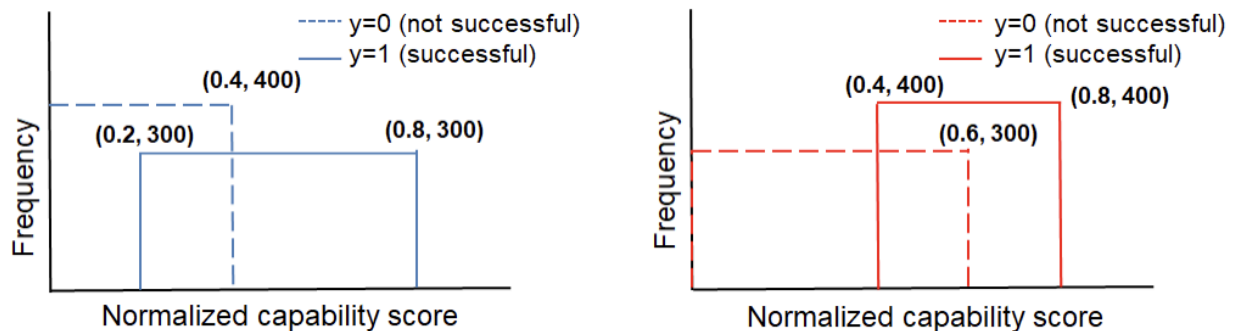
The final model will provide us with an interpretable approximation around the point of interest (10, 12). Would you like me to calculate the linear model coefficients, or is there any specific part you'd like to focus on?

Topic: Fairness Detection Metric

1. Compute demographic parity, equality of odds and equality of opportunity for a given dataset. Refer problems discussed in lectures 2 and 3.

Topic: Fairness Detection Metrics

2. A university that uses normalized capability score for approving or rejecting an applicant. Note there are two population of applicants 'blue' (deprived group with protected attribute $A=0$) and 'red' (favored group with $A=1$). Admission is approved, i.e., $y'=1$ if normalized capability score is greater than some threshold t , else rejected. $y=1$ indicates that an admitted student graduates successfully and timely, and $y=0$ indicates admitted student was not successful. The university wants to admit applicants that will graduate successfully and timely. Distributions are described in following figure (not drawn to scale) for both populations.



- i. Calculate probability $P(y'=1)$ for both 'blue' and 'red' populations for threshold $t=0.4$.

For 'blue', $P(y'=1) = (0.8-0.4)300/[0.4*400+0.6*300]=12/34=6/17$

For 'red' $P(y'=1) = [(0.8-0.4)*400+(0.6-0.4)300] / [0.4*400+0.6*300] = 22/34=11/17$

- i. If threshold $t=0.4$ is chosen for the blue population, what threshold value for the red population will ensure demographic parity?

For $t=0.4$, $P(y'=1)=6/17$

Let $t_{red} < 0.6$ be the desired threshold for red.

So, $(0.8-t_{red})400 + (0.6-t_{red})300=120 \rightarrow t_{red} = 380/700 = 19/35$

- i. If threshold $t=0.6$ is chosen for the blue population, calculate the probability $P(y'=1|y=1)$. Is there a threshold for the red population that will ensure equal opportunity? Show mathematically. For the same threshold value, is equality of odds achieved? Why?

If $t=0.6$, for blue population $P(y'=1|y=1) = (0.8-0.6)300/180=1/3$

Let t_{red} be the threshold for red population that ensures Equal Opportunity.

Then, $(0.8-t_{red})400=0.4*400/3 \rightarrow t_{red}=0.8-0.4/3=2/3$

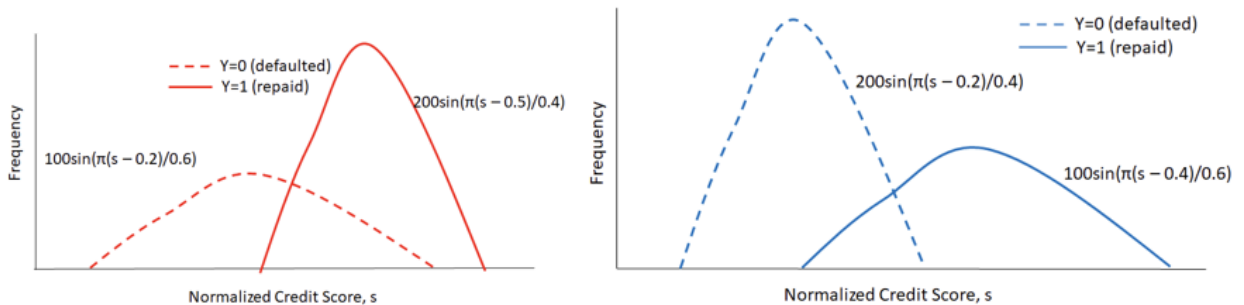
Now, for blue population, at $t=0.6$, $P(y'=1|y=0) = 60/160=3/8$

For red population, at $t_{red}=2/3$, $P(y'=1|y=0)=52/180$

So, Equality of odds cannot be achieved.

3. Consider a financial institution that uses normalized credit score for approving or rejecting housing loan. Note there are two population of applicants 'blue' (deprived group with protected attribute $A=0$) and 'red' (favored group with $A=1$). Loan is approved, i.e., $y'=1$ if normalized credit score is greater than some threshold t , else rejected. $y=1$ indicates approved loans that are repaid based on historical data, and $y=0$ indicates approved loan was defaulted. The financial institution wants to approve loans that is likely to be repaid. Distributions are described in following figure (not drawn to scale) for both populations.

Note: $\int \sin(ax) dx = -\cos(ax)/a$



- A. Calculate probability $P(y'=1)$ for both 'blue' and 'red' populations for threshold $t=0.4$.

For blue, $P(y'=1) = (100+100)/300=0.67$

For red, $P(y'=1)=100(1+\cos(\pi/3))+200*2/600 = 0.916$

- A. If threshold $t=0.4$ is chosen for the blue population, what threshold value, if any for the red population will ensure demographic parity?

At $t=0.4$, for blue, $P(y'=1)=.67$

Let at $t=t_{red}$ $P(y')=0.67$

Then, $200(1+\cos(\pi(t_{red}-0.5)/0.4)) + 100(1 + \cos(\pi(t_{red} - 0.2)/0.6)) = 400$

Or, $2 \cos(\pi(t_{red}-0.5)/0.4) + \cos(\pi(t_{red}- 0.2)/0.6) = 1$

Or, $t_{red} = 0.59435$

- A. If threshold $t=0.6$ is chosen for the blue population, calculate the probability $P(y'=1|y=1)$. What threshold value for the red population will ensure equal opportunity? For the same threshold value, calculated the probability of $P(y'=1|y=0)$ for both blue and red populations. Is equality of odds achieved?

At $t=0.6$, $P(y'=1|y=1) = (1+\cos(\pi/3))/2=0.75$ for blue

For red, $t=0.6534$ $P(y'=1|y=1) = 0.75$

[complete the $P(y'=1|y=0)$ for both blue and red and compare]

Topic: Fair ML Methods In-processing

4. Consider the following training set. Gender is the sensitive attribute, P is the predictor and C is the target class.

Gender	P	C
F	8	-
F	9	-
F	11	+
M	9.5	-
M	12	+
M	13	+

A. Calculate the Prejudice Index (PI) and Normalized Prejudice Index (NPI) of the dataset.

$$\begin{aligned}
 &Pr[-,F]=1/3 \quad Pr[+,F]=1/6 \quad Pr[-,M]=1/6 \quad Pr[+,M]=1/3 \\
 &Pr[-]=0.5 \quad Pr[+]=0.5 \quad Pr[F]=0.5 \quad Pr[M]=0.5 \\
 &So, PI = 0.33 \ln(0.33/0.25) + 0.167 \ln(0.167/0.25) + 0.167 \ln(0.167/0.25) + 0.33 \ln(0.33/0.25) \\
 &\quad = 0.67 \ln(0.33/0.25) + 0.33 \ln(0.167/0.25) = 0.053 \\
 &H(S)=H(Y)=1, so NPI = PI
 \end{aligned}$$

B. After training with Prejudice Removal Regularization, a logistic regression based classifier is obtained as $\sigma(\text{Gender}, P) = 1/(1+\exp(9-P+2*\text{Gender}))$. F is encoded as 0 and M is encoded as 1 while training the classifier.

i. What is the NPI value for the classifier? Is discrimination improved? Why?

Gender	P	C	Predicted Class	Actual Output M
F	8	-	-	$1/(1+e^1)=0.27$
F	9	-	+	0.5
F	11	+	+	$1/(1+e^2)=0.88$
M	9.5	-	-	$1/(1+e^{1.5})=0.18$
M	12	+	+	$1/(1+e^{-1})=0.73$
M	13	+	+	$1/(1+e^2)=0.88$

$$\begin{aligned}
 &Pr[y] = (0.27+0.5+0.88+0.18+0.73+0.88)/6=0.57 \\
 &Pr[y|F] = (0.27+0.5+0.88)/3=0.55 \\
 &Pr[y|M] = (0.18+0.73+0.88)/3=0.597 \\
 &PI = (0.27+0.5+0.88) \ln(0.55/0.57) + (0.18+0.73+0.88) \ln(0.597/0.57) = 0.024 \\
 &NPI=0.024
 \end{aligned}$$

Discrimination improves since NPI is less than in the original data!

ii. What is the accuracy of the classifier? Assume predicted C =1 if classifier output ≥ 0.5 .

$$Accuracy = \%$$

5. Consider the following training set. Gender is the sensitive attribute, Skill and Years are predictors and Y is the target decision.

Gender	Skill	Years of Experience	Promotion Decision Y
Female	Java	2	no
Female	C++	5	yes
Male	Java	2	yes
Male	C++	3	yes

A. Calculate the Prejudice Index (PI) and Normalized Prejudice Index (NPI) of the dataset.

$$\begin{aligned}
 \Pr[-,F] &= 1/4 & \Pr[+,F] &= 3/4 & \Pr[-,M] &= 0 & \Pr[+,M] &= 1/2 \\
 \Pr[-] &= 0.25 & \Pr[+] &= 0.75 & \Pr[F] &= 0.5 & \Pr[M] &= 0.5 \\
 \text{So, PI} &= \frac{3}{4} \ln(3/4/0.75*0.5) + \frac{1}{4} \ln(1/4/0.25*0.5) + \frac{1}{2} \ln(1/2/0.75*0.5) \\
 &= \frac{3}{4} \ln(2) + \frac{1}{4} \ln(2) + \frac{1}{2} \ln(4/3) \\
 H(S) &= 1 & H(Y) &= \frac{1}{4} \ln(1/4) + \frac{3}{4} \ln(3/4), \text{ so NPI} = \text{PI}
 \end{aligned}$$

A. A logistic regression based classifier with prejudice removal regularizer is trained to obtain fair decisions. All classifier parameters except the one associated with Gender are initialized to 0. Parameter w_{gender} associated with Gender is initialized to 1. Female and C++ are encoded as 0 and Male and Java are encoded as 1 while training the classifier. Assuming learning rate = 0.1 and regularization constant is 0.5, calculate the value of classifier parameters after 1st iteration without and with prejudice removal regularizer. Assume batch training is being used. Show all calculation steps.

Gender	Skill	Years of Experience	Promotion Decision Y	initial Output M
0	1	2	no	0.5
0	0	5	yes	0.5
1	1	2	yes	$1/(1+e^{-1})$ =0.731
1	0	3	yes	$1/(1+e^{-1})$ =0.731

So, in the next iteration without regularization,

$$\begin{aligned}
 w_0 &= 0 - 0.1/4 * ((0.5) + (0.5-1) + (1-0.731) + (1-0.731)) = 0.013 \\
 w_{\text{gender}} &= 1 - 0.1/4 * ((1-0.731) + (1-0.731)) = 1.013 \\
 w_{\text{skill}} &= 0 - 0.1/4 * (0.5 + (1-0.731)) = -0.006 \\
 w_{\text{exp}} &= 0 - 0.1/4 * (0.5*2 + (0.5-1)*5 + (0.731-1)*2 + (0.731-1)*3) = 0.071
 \end{aligned}$$

If PI removal regularization is used, regularization loss R to be optimized is given by

Gender	Skill	Years of Experience	Promotion Decision Y	initial Output M	$\ln(\Pr[y s_i]/\Pr[y])$
--------	-------	---------------------	----------------------	------------------	--------------------------

0	1	2	no	0.5	$\ln(0.5/0.6155)=-0.21$
0	0	5	yes	0.5	$\ln(0.5/0.6155)=-0.21$
1	1	2	yes	$1/(1+e^{-1})=0.731$	$\ln(0.731/0.6155)=.17$
1	0	3	yes	$1/(1+e^{-1})=0.731$	$\ln(0.731/0.6155)=0.17$

After regularization is applied,

$$w_0 = 0.013 - 0.5 * 0.1/4 * ((0.5) * (-0.21) + (0.5-1) * (-0.21) + (0.731-1) * 0.17 + (0.731-1) * 0.17) = 0.0146$$

$$w_{\text{gender}} = 1.013 - 0.5 * 0.1/4 * ((0.5) * (-0.21) * 0 + (0.5-1) * (-0.21) * 0 + (0.731-1) * 0.17 + (0.731-1) * 0.17) = 1.013$$

$$\text{Similarly, } w_{\text{skin}} = -0.0062 \text{ and } w_{\text{exp}} = 0.072$$

Topic: Fair ML Methods Pre-processing: Universal Sampling

6. Apply universal sampling to generate a training dataset from the following dataset. Gender is the protected attribute, P is the predictor and C is the target class. How many distinct datasets can be created?

Gender	P	C
F	6	-
F	8	-
F	9	-
F	11	+
M	9.5	-
M	12	+
M	13	+
M	16.5	+

One possible training set after sampling will be

Gender	P	C
F	8	-
F	9	-
F	11	+
F	11	+
M	9.5	-
M	9.5	-
M	12	+
M	13	+

(F, -) and (M, +) need to be undersampled from 3 (in the original dataset) to 2 instances (after sampling). Since there are 3 instances of (F, -) and (M, +) in the original set, total number of distinct training set that can be created will be $3 \times 3 = 9$. The dataset shown above is one such training set.

7. Apply preferential sampling to create the training data from the following dataset. Gender is the protected attribute, P is the predictor and C is the target class. A logistic regression ranker $r(P)=1/(1+\exp(10-P))$ is used.

Gender	P	C
F	6	-
F	8	-
F	9	-
F	11	+
M	9.5	-
M	12	+
M	13	+
M	16.5	+

Gender	P	R(P)	C
F	6	$1/(1+e^4)$	-
F	8	$1/(1+e^2)$	-
F	9	$1/(1+e)$	-
F	11	$1/(1+e^{-1})$	+
M	9.5	$1/(1+e^{0.5})$	-
M	12	$1/(1+e^{-2})$	+
M	13	$1/(1+e^{-3})$	+
M	16.5	$1/(1+e^{-6.5})$	+

After applying preferential sampling, the new training set will be

Gender	P	C
F	8	-
F	9	-
F	11	+
F	11	+
M	9.5	-
M	9.5	-
M	12	+
M	13	+

Assume a logistic regression based classifier is used to classify the following dataset while ensuring fairness. Gender is the protected attribute.

Gender	Skill	Years of Experience	Goes to the Gynecologist?	Promotion Decision Y
Female	Java	1	yes	no
Female	C++	5	yes	yes
Male	Java	2	no	yes
Male	C++	3	no	yes

- i. If reweighting based preprocessing is used, what will be the reweighting factor for the given data records?

Gender	Skill	Years of Experience	Goes to the Gynecologist?	Promotion Decision Y
Female	Java	1	yes	no
Female	C++	5	yes	yes
Male	Java	2	no	yes
Male	C++	3	no	yes

$$w_1 = 0.5 \cdot 0.25 / 0.25 = .5$$

$$w_2 = 0.5 \cdot 0.75 / 0.25 = 1.5$$

$$w_3 = 0.5 \cdot 0.75 / 0.5 = 0.75$$

$$w_4 = 0.75$$

- i. The parameter vector of the classifier is initialized to **0**. If needed, assume C++, and Java are represented by 0 and 1, yes by 1 and no by 0, Female by 0 and Male by 1. What will be the parameter vector after 1st iteration, if learning rate is 0.1?

Original parameter update for logistic regression is given by

With reweighting based preprocessing, the update equation needs to be modified using weights $w_1, \dots, w_4$ as

Gender	Skill	Years of Experience	Goes to the Gynecologist?	Promotion Decision y	Initial Output
Female	1	1	1	0	0.5
Female	0	5	1	1	0.5
Male	1	2	0	1	0.5
Male	0	3	0	1	0.5

$$\theta_0 = -0.1/4 \cdot [0.5 \cdot 0.5 - 1.5 \cdot 0.5 - 0.75 \cdot .5 - 0.75 \cdot 0.5] = 0.025 \cdot 1.25$$

$$\theta_{\text{gender}} = -0.1/4 \cdot [0.5 \cdot 0.5 \cdot 0 - 1.5 \cdot 0.5 \cdot 0 - 0.75 \cdot 0.5 - 0.75 \cdot 0.5] = 0.025 \cdot 0.75$$

$$\theta_{\text{skill}} = -0.1/4 \cdot [0.5 \cdot 0.5 \cdot 1 - 1.5 \cdot 0.5 \cdot 0 - 0.75 \cdot 0.5 - 0.75 \cdot 0.5 \cdot 0] = 0.025 \cdot 0.35$$

$$\theta_{\text{exp}} = -0.1/4 \cdot [0.5 \cdot 0.5 \cdot 1 - 1.5 \cdot 0.5 \cdot 5 - 0.75 \cdot 0.5 \cdot 2 - 0.75 \cdot 0.5 \cdot 3] = 0.025 \cdot 5.375$$

$$\theta_{\text{gyno}} = -0.1/4 \cdot [0.5 \cdot 0.5 \cdot 1 - 1.5 \cdot 0.5 \cdot 1 - 0.75 \cdot 0.5 \cdot 0 - 0.75 \cdot 0.5 \cdot 0] = 0.025 \cdot 0.5$$

Topic: Word Embedding, Bag of Words

<https://openclassrooms.com/en/courses/6532301-introduction-to-natural-language-processing/8081284-apply-a-simple-bag-of-words-approach>

Topic: Fair NLP - Lecture 8

8. Consider gender debiasing regularization for word embeddings. The following are the word embeddings. $W_g(.)$ and $w_g(.)$ are the gender-related and gender-neutral parts.

Word	Male seed words	Female seed words	$w_g(1)$	$w_g(2)$	$w_g(3)$	$w_a(1)$	$w_a(2)$
daughter		X	0.3	0.5	0.6	0.3	0.6
son	X		.75	0.6	0.3	0.6	0.5
prince	X		0.75	0.6	0.3	0.3	0.5
princess		X	0.3	0.5	0.6	0.75	0.6
doctor			0.4	0.3	0.8	0.8	0.1
lawyer			0.2	0.1	0.8	0.4	0.3

- i. Calculate the gendered word regulating component J_d .

$$\begin{aligned} J_d &= \|[.75*2 \ 0.6*2 \ 0.3*2] - [0.3*2 \ 0.5*2 \ 0.6*2]\|_1 \\ &\quad + 2\|[1-0.75 \ 1-0.6 \ 1-0.3]\|_2^2 + 2\|[-1-0.3 \ -1-0.5 \ -1-0.6]\|_2^2 \\ &= 1.7+1.425+13=16.125 \end{aligned}$$

- i. Calculate the gender-neutral word regulating component J_e .

and

$$\begin{aligned} v_g &= \frac{1}{4} * ([0.6 \ 0.5] - [.3 \ .6] + [0.6 \ 0.5] - [.75 \ .6] + [0.3 \ 0.5] - [.3 \ .6] + [0.3 \ 0.5] - [.75 \ .6]) \\ &= \frac{1}{4} * [-0.3 \ 0.4] \end{aligned}$$

$$J_e = \frac{1}{4} * (-.8*.3 + 0.1*0.4 - 0.4*0.3 + 0.3*0.4) = 0.05$$

Q.1.

- A. Apply preferential sampling to create the training data from the following dataset. Gender is the protected attribute, P is the predictor and C is the target class. A logistic regression ranker $r(P)=1/(1+\exp(10-P))$ is used.

Gender	P	C
F	6	-
F	8	-
F	9	-
F	11	+
M	9.5	-
M	12	+
M	13	+
M	16.5	+

Gender	P	R(P)	C
F	6	$1/(1+e^4)$	-
F	8	$1/(1+e^2)$	-
F	9	$1/(1+e^1)$	-
F	11	$1/(1+e^{-1})$	+
M	9.5	$1/(1+e^{0.5})$	-
M	12	$1/(1+e^{-2})$	+
M	13	$1/(1+e^{-3})$	+
M	16.5	$1/(1+e^{-6.5})$	+

After applying preferential sampling, the new training set will be

Gender	P	C
F	8	-
F	9	-
F	11	+
F	11	+
M	9.5	-
M	9.5	-
M	12	+
M	13	+

- B. Assume a logistic regression based classifier is used to classify the following dataset while ensuring fairness. Gender is the protected attribute.

Gender	Skill	Years of Experience	Goes to the Gynecologist?	Promotion Decision Y
Female	Java	1	yes	no
Female	C++	5	yes	yes
Male	Java	2	no	yes
Male	C++	3	no	yes

- i) If reweighting based preprocessing is used, what will be the reweighting factor for the given data records?

Gender	Skill	Years of Experience	Goes to the Gynecologist?	Promotion Decision Y
Female	Java	1	yes	no
Female	C++	5	yes	yes
Male	Java	2	no	yes
Male	C++	3	no	yes

$$w_1 = 0.5 * 0.25 / 0.25 = .5$$

$$w_2 = 0.5 * 0.75 / 0.25 = 1.5$$

$$w_3 = 0.5 * 0.75 / 0.5 = 0.75$$

$$w_4 = 0.75$$

- ii) The parameter vector of the classifier is initialized to **0**. If needed, assume C++, and Java are represented by 0 and 1, yes by 1 and no by 0, Female by 0 and Male by 1. What will be the parameter vector after 1st iteration, if learning rate is 0.1?

Original parameter update for logistic regression is given by

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

With reweighing based preprocessing, the update equation needs to be modified using weights $w_1, \dots, w_4$ as

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m w_m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Gender	Skill	Years of Experience	Goes to the Gynecologist?	Promotion Decision y	Initial Output
Female	1	1	1	0	0.5
Female	0	5	1	1	0.5
Male	1	2	0	1	0.5
Male	0	3	0	1	0.5

$$\theta_0 = -0.1/4 * [0.5*0.5 - 1.5*0.5 - 0.75*0.5 - 0.75*0.5] = 0.025*1.25$$

$$\theta_{\text{gender}} = -0.1/4 * [0.5*0.5*0 - 1.5*0.5*0 - 0.75*0.5 - 0.75*0.5] = 0.025*0.75$$

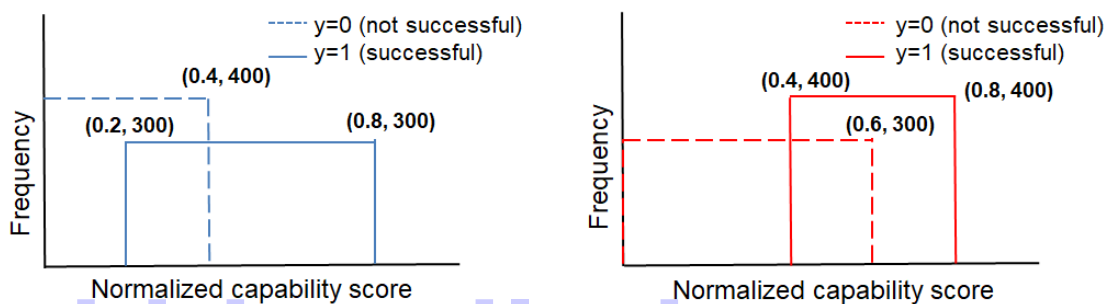
$$\theta_{\text{skill}} = -0.1/4 * [0.5*0.5*1 - 1.5*0.5*0 - 0.75*0.5 - 0.75*0.5*0] = 0.025*0.35$$

$$\theta_{\text{exp}} = -0.1/4 * [0.5*0.5*1 - 1.5*0.5*5 - 0.75*0.5*2 - 0.75*0.5*3] = 0.025*5.375$$

$$\theta_{\text{gyno}} = -0.1/4 * [0.5*0.5*1 - 1.5*0.5*1 - 0.75*0.5*0 - 0.75*0.5*0] = 0.025*0.5$$

Q.2.

A university that uses normalized capability score for approving or rejecting an applicant. Note there are two population of applicants 'blue' (deprived group with protected attribute $A=0$) and 'red' (favored group with $A=1$). Admission is approved, i.e., $y'=1$ if normalized capability score is greater than some threshold t , else rejected. $y=1$ indicates that an admitted student graduates successfully and timely, and $y=0$ indicates admitted student was not successful. The university wants to admit applicants that will graduate successfully and timely. Distributions are described in following figure (not drawn to scale) for both populations.



- i) Calculate probability $P(y'=1)$ for both 'blue' and 'red' populations for threshold $t=0.4$.
 For 'blue', $P(y'=1) = (0.8-0.4)300/[0.4*400+0.6*300]=12/34=6/17$
 For 'red' $P(y'=1) = [(0.8-0.4)*400+(0.6-0.4)300] / [0.4*400+0.6*300] = 22/34=11/17$
- ii) If threshold $t=0.4$ is chosen for the blue population, what threshold value for the red population will ensure demographic parity?
 For $t=0.4$, $P(y'=1)=6/17$
 Let $t_{red} < 0.6$ be the desired threshold for red.
 So, $(0.8-t_{red})400 + (0.6-t_{red})300=120 \rightarrow t_{red} = 380/700 = 19/35$
- iii) If threshold $t=0.6$ is chosen for the blue population, calculate the probability $P(y'=1|y=1)$. Is there a threshold for the red population that will ensure equal opportunity? Show mathematically. For the same threshold value, is equality of odds achieved? Why?
 If $t=0.6$, for blue population $P(y'=1|y=1) = (0.8-0.6)300/180=1/3$
 Let t_{red} be the threshold for red population that ensures Equal Opportunity.
 Then, $(0.8-t_{red})400=0.4*400/3 \rightarrow t_{red}=0.8 - 0.4/3 = 2/3$
 Now, for blue population, at $t=0.6$, $P(y'=1|y=0) = 60/160=3/8$
 For red population, at $t_{red}=2/3$, $P(y'=1|y=0)=52/180$
 So, Equality of odds cannot be achieved.

Q.3.

Consider the following training set. Gender is the sensitive attribute, P is the predictor and C is the target class.

- A. Calculate the Prejudice Index (PI) and Normalized Prejudice Index (NPI) of the dataset.

$$PI = \sum_{(y,s) \in \mathcal{D}} \hat{P}_r[y, s] \ln \frac{\hat{P}_r[y, s]}{\hat{P}_r[y] \hat{P}_r[s]}$$

$$NPI = PI / (\sqrt{H(Y)H(S)})$$

$$\begin{array}{llll} \Pr[-,F]=1/3 & \Pr[+,F]=1/6 & \Pr[-,M]=1/6 & \Pr[+,M]=1/3 \\ \Pr[-]=0.5 & \Pr[+]=0.5 & \Pr[F]=0.5 & \Pr[M]=0.5 \end{array}$$

$$\begin{aligned} \text{So, } PI &= 0.33 \ln(0.33/0.25) + 0.167 \ln(0.167/0.25) + 0.167 \ln(0.167/0.25) + 0.33 \ln(0.33/0.25) \\ &= 0.67 \ln(0.33/0.25) + 0.33 \ln(0.167/0.25) = 0.053 \end{aligned}$$

$$H(S)=H(Y)=1, \text{ so } NPI = PI$$

- B. After training with Prejudice Removal Regularization, a logistic regression based classifier is obtained as $\sigma(\text{Gender}, P) = 1/(1+\exp(9-P+2*\text{Gender}))$. F is encoded as 0 and M is encoded as 1 while training the classifier.

- i) What is the NPI value for the classifier? Is discrimination improved? Why?

Gender	P	C	Predicted Class	Actual Output $\mathcal{M}$
F	8	-	-	$1/(1+e^1)=0.27$
F	9	-	+	0.5
F	11	+	+	$1/(1+e^{-2})=0.88$
M	9.5	-	-	$1/(1+e^{1.5})=0.18$
M	12	+	+	$1/(1+e^{-1})=0.73$
M	13	+	+	$1/(1+e^{-2})=0.88$

$$\hat{\Pr}[y|s] \approx \frac{\sum_{(\mathbf{x}_i, s_i) \in \mathcal{D} \text{ s.t. } s_i=s} \mathcal{M}[y|\mathbf{x}_i, s; \boldsymbol{\Theta}]}{|\{(\mathbf{x}_i, s_i) \in \mathcal{D} \text{ s.t. } s_i=s\}|}$$

$$\hat{\Pr}[y] \approx \frac{\sum_{(\mathbf{x}_i, s_i) \in \mathcal{D}} \mathcal{M}[y|\mathbf{x}_i, s_i; \boldsymbol{\Theta}]}{|\mathcal{D}|}$$

$$\text{PI} = \sum_{(\mathbf{x}_i, s_i) \in \mathcal{D}} \sum_{y \in \{0,1\}} \mathcal{M}[y|\mathbf{x}_i, s_i; \boldsymbol{\Theta}] \ln \frac{\hat{\Pr}[y|s_i]}{\hat{\Pr}[y]}$$

$$\Pr[y] = (0.27+0.5+0.88+0.18+0.73+0.88)/6=0.57$$

$$\Pr[y|F] = (0.27+0.5+0.88)/3=0.55$$

$$\Pr[y|M] = (0.18+0.73+0.88)/3=0.597$$

$$\text{PI} = (0.27+0.5+0.88)\ln(0.55/0.57) + (0.18+0.73+0.88)\ln(0.597/0.57) = 0.024$$

$$\text{NPI} = 0.024$$

Discrimination improves since NPI is less than in the original data!

- ii) What is the accuracy of the classifier? Assume predicted C =1 if classifier output ≥ 0.5 .

$$\text{Accuracy} = 5/6$$

Gender	P	C
F	8	-
F	9	-
F	11	+
M	9.5	-
M	12	+
M	13	+

Q.1.

- A. Consider DP-SGD is being used to train a logistic regression based classifier of the form, $\text{output} = 1/(1+\exp(-w_0-w_1*x_1-w_2*x_2))$. All parameters w_0 , w_1 and w_2 of the classifier are initialized to 0. Mini-batch size is chosen to be 4, and gradient norm bound is chosen to be 0.6. What will be the expected value of gradient vector after first iteration? Show all calculation steps clearly.

Data Instance #	Input x1	Input x2	Target y
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1

				w0	w1	w2			
				0	0	0			
Data Instance #	Input x1	Input x2	Target y	Actual Output	dL/dw0	dL/dw1	dL/dw2	L2 norm of gradient	Clipped L2 norm
1	0	0	0	0.5	0.5	0	0	0.5	0.5
2	0	1	0	0.5	0.5	0	0.5	0.7071	0.6
3	1	0	0	0.5	0.5	0.5	0	0.7071	0.6
4	1	1	1	0.5	-0.5	-0.5	-0.5	0.8660	0.6

0.575 average Gra

- B. Assume federated learning setting with the above training set. Data instance # 1 and #4 are processed by first node and #2 and #3 are processing by the second node. As in A., parameters are initialized to 0 and learning rate is 0.6. What will be the value of parameters after 1st iteration if
- i) FedSGD is used?

FedSGD

$$g_k = \nabla F_k(w_t)$$

$$w_{t+1} \leftarrow w_t - \eta \sum_{k=1}^K \frac{n_k}{n} g_k$$

Data Instance #	Input x1	Input x2	Target y	Actual Output	dL/dw0	dL/dw1	dL/dw2
1	0	0	0	0.5	0.5	0	0
2	0	1	0	0.5	0.5	0	0.5
3	1	0	0	0.5	0.5	0.5	0
4	1	1	1	0.5	-0.5	-0.5	-0.5

Node 1

weight change

w0

w1	w2
0.075	0.075

Node 2 weight change w0,w1,w2

New weight

-0.15 -0.075 -0.075

-0.15 0 0

- ii) FedAvg is used?

FedAvg

$$w_{t+1}^k \leftarrow w_t - \eta g_k$$

$$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$$

Data Instance #	Input x1	Input x2	Target y	Actual Output	dL/dw0	dL/dw1	dL/dw2
1	0	0	0	0.5	0.5	0	0
2	0	1	0	0.5	0.5	0	0.5
3	1	0	0	0.5	0.5	0.5	0
4	1	1	1	0.5	-0.5	-0.5	-0.5

Node 1	Node 1	Node 1	Node 2	Node 2	Node 2	New Weights		
Weight	Weight	Weight	Weight	Weight	Weight			
w0	w1	w1	w0	w1	w2			
0	0.15	0.15	-0.3	-0.15	-0.15	-0.15	0	0

Q.2

- A. Consider gender debiasing regularization for word embeddings. The following are the word embeddings. $w_g(\cdot)$ and $w_a(\cdot)$ are the gender-related and gender-neutral parts.

Word	Male seed words	Female seed words	$w_g(1)$	$w_g(2)$	$w_g(3)$	$w_a(1)$	$w_a(2)$
daughter		X	0.3	0.5	0.6	0.3	0.6
son	X		.75	0.6	0.3	0.6	0.5
prince	X		0.75	0.6	0.3	0.3	0.5
programmer			0.3	0.2	0.6	0.6	0.2
doctor			0.4	0.3	0.8	0.8	0.1
lawyer			0.2	0.1	0.8	0.4	0.3

- i) Calculate the gendered word regulating component J_D .

$$J_D = - \left\| \sum_{w \in \Omega_M} w^{(g)} - \sum_{w \in \Omega_F} w^{(g)} \right\|_1 + \sum_{w \in \Omega_M} \left\| 1 - w^{(g)} \right\|_2^2 + \sum_{w \in \Omega_F} \left\| -1 - w^{(g)} \right\|_2^2$$

$$J_D = - \left\| [0.75 \cdot 2 \quad 0.6 \cdot 2 \quad 0.3 \cdot 2] - [0.3 \quad 0.5 \quad 0.6] \right\|_1 + 2 \left\| [1 - 0.75 \quad 1 - 0.6 \quad 1 - 0.3] \right\|_2^2 + \left\| [-1 - 0.3 \quad -1 - 0.5 \quad -1 - 0.6] \right\|_2^2$$

=

- ii) Calculate the gender-neutral word regulating component J_E .

$$J_E = \sum_{w \in \Omega_N} \left(v_g^T w^{(a)} \right)^2$$

and

$$v_g = \frac{1}{|\Omega'|} \sum_{(w_m, w_f) \in \Omega_M, \Omega_F} (w_m^{(a)} - w_f^{(a)})$$

$$v_g = 1/2 * ([0.6 \quad 0.5] - [.3 \quad .6] + [0.3 \quad 0.5] - [.3 \quad .6])$$

=

$$J_E = 1/4 * (-.8 * .3 + 0.1 * 0.4 - 0.4 * 0.3 + 0.3 * 0.4) = 0.05$$

- B. Consider a logistic regression based classifier being trained with the usual log-loss function on the dataset shown below. At a particular iteration, all the parameters of the network have the value 0.5. Batch size is chosen to be 4.
- If one-step untargeted Fast Gradient Sign Method is used, calculate the adversarial input sample. $\epsilon = 0.6$.
 - If one-step targeted Fast Gradient Sign Method is used, calculate the adversarial input sample. Data Instance #1 is targeted, i.e., target y for instance #1 is set to 1. Rest of the true values are unchanged. $\epsilon = 0.6$.

Data Instance #	Input x1	Input x2	True y
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1

$$\mathbf{X}^{adv} = \mathbf{X} + \epsilon \text{sign}(\nabla_{\mathbf{X}} J(\mathbf{X}, y_{true}))$$

Untargeted FGSM

Data Instance #	Input x1	Input x2	True y	Actual y	dLoss/dw1	dLoss/dw2	x1_adv	x2_adv
1	0	0	0	0.622459	-0.31123	-0.31123	-0.6	-0.6
2	0	1	0	0.731059	-0.36553	-0.36553	-0.6	0.4
3	1	0	0	0.731059	-0.36553	-0.36553	0.4	-0.6
4	1	1	1	0.817574	0.091213	0.091213	1.6	1.6

$$\text{Loss} = -\text{True } y \cdot \ln(\text{actual } y) - (1 - \text{True } y) \cdot \ln(1 - \text{actual } y)$$

$$d\text{Loss}/dx1 = (\text{true } y - \text{actual } y) \cdot w1$$

$$d\text{Loss}/dx2 = (\text{true } y - \text{actual } y) \cdot w2$$

$$\mathbf{X}^{adv} = \mathbf{X} - \epsilon \text{sign}(\nabla_{\mathbf{X}} J(\mathbf{X}, y_{target}))$$

Targeted FGSM

Data Instance #	Input x1	Input x2	Target y	Actual y	dLoss/dw1	dLoss/dw2	x1_adv	x2_adv
1	0	0	1	0.622459	0.18877	0.18877	-0.6	-0.6
2	0	1	0	0.731059	-0.36553	-0.36553	0.6	1.6
3	1	0	0	0.731059	-0.36553	-0.36553	1.6	0.6
4	1	1	1	0.817574	0.091213	0.091213	0.4	0.4

$$\text{Loss} = -\text{Target } y \cdot \ln(\text{actual}) - (1 - \text{Target } y) \cdot \ln(1 - \text{actual } y)$$

$$d\text{Loss}/dx1 = (\text{target } y - \text{actual } y) \cdot w1$$

$$d\text{Loss}/dx2 = (\text{target } y - \text{actual } y) \cdot w2$$

Note: Batch size is not relevant for the problem. Full marks can be awarded if only one adversarial sample is correctly generated as long as the corresponding data instance is clearly referred to.

Q.3. Consider the following training dataset relating features x_1 (sq. ft. of a flat) and x_2 (distance of the flat from the city centre) and its price y . Linear regression model is used. No scaling is used.

Instance #	x_1 (sq. ft.)	x_2 (distance in km)	Y (price in lakhs of rupees)
1	500	1	25
2	750	2	35
3	800	3	35
4	1200	5	50

Calculate the standard Shapley values for a flat of 1000 sq ft. and at a distance of 4 km. Show all steps including all the models obtained for different coalitions of features, marginal contributions clearly.

$$\text{Model}(\text{NULL}) = (25+35+35+50)/4=36.25$$

$$\text{Model}(x_1) = 7.419 + 0.0355x_1$$

$$\text{Model}(x_2) = 20.143 + 5.857x_2$$

$$\text{Model}(x_1, x_2) = 2.5 + 0.05x_1 - 2.5x_2$$

At $(x_1, x_2) = (1000, 4)$, prediction using

$$\text{model}(\text{null}) = 36.25$$

$$\text{model}(x_1) = 7.419 + 0.0355 * 1000 = 42.919$$

$$\text{model}(x_2) = 20.143 + 5.857 * 4 = 43.571$$

$$\text{model}(x_1, x_2) = 2.5 + 0.05 * 1000 - 2.5 * 4 = 42.5$$

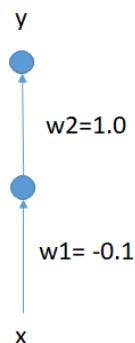
$$\text{Contribution due to } x_1 = 0.5 * (42.919 - 36.25 + 42.5 - 43.571) = 2.799$$

$$\text{Contribution due to } x_2 = 0.5 * (43.571 - 36.25 + 42.5 - 42.919) = 3.451$$

Q.1.

- A. Consider DP-SGD is being used to train the following neural network. All biases are initialized to 0 and weights w_1 and w_2 are initialized as shown. Sigmoid activation function is used. Mini-batch size is chosen to be 2, and gradient norm bound is chosen to be 0.6. What will be the expected value of gradient vector after first iteration? Show all calculation steps clearly. Note gradient vector comprises of both biases and weights. Usual binary cross-entropy is used as the loss function.

Data Instance #	Input x	Target y
1	0	1
2	1	0

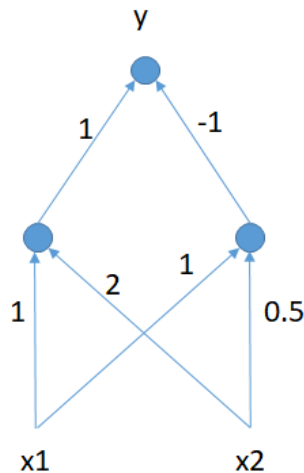


Input x	Target y	Hidden O	Actual O
0	1	0.5	0.622459
1	0	0.475021	0.616571

gradient b2	gradient w2	gradient b1	gradient w1	L2 Norm	Clipped Gradient
-0.37754	-0.18877	-0.09439	0	0.432527	0.432527
0.616571	0.292884	0.153758	0.153758	0.716397	0.6
					Average
					0.516264 Gradient

- B. Calculate the saliency values for the input nodes for data instance #2 and #4 w.r.t. the actual output for the trained neural network shown below. All biases are 0, ReLU activation is used the hidden node and sigmoid at the output node. Show all steps clearly. Assume derivative of ReLU(x) at x=0 is 1, if needed.

Data Instance #	Input x1	Input x2	True y
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1



Input x1	Input x2	Left hidden output h1	Right hidden output h2	Actual y	dy/dx1	dy/dx2
0	1	2	0.5	0.817574	0	0.22372
1	1	3	1.5	0.817574	0	0.22372

Note: derivative of ReLU is 1 is derivative of sigmoid function $f(x)$ w.r.t. x is $f(x)(1-f(x))$

$$dy/dx1 = y(1-y)*(1*1-1*1)$$

$$dy/dx2 = y(1-y)*(1*2-1*0.5)$$

Q.2

- A. Apply preferential sampling to create the training data from the following dataset. Gender is the protected attribute, P is the predictor and C is the target class. The following neural net based ranker is used. Hidden bias is 1 and output bias is 0, sigmoid activation is used.

Gender	P	C
F	6	-
F	8	-
F	10	-
F	11	+
M	9.5	-
M	10.5	+
M	12	+
M	13	+



P	C	hidden input	hidden output	final output / ranker output	Action
6	-	0.4	0.598688	0.645356	Remove
8	-	0.2	0.549834	0.634097	Keep
10	-	0	0.5	0.622459	Keep
11	+	-0.1	0.475021	0.616571	Duplicate
9.5	-	0.05	0.512497	0.625392	Duplicate
10.5	+	-0.05	0.487503	0.619518	Keep
12	+	-0.2	0.450166	0.610679	Keep
13	+	-0.3	0.425557	0.604812	Remove

- B. Consider the following trained neural network classifier with usual binary cross-entropy loss function on the following dataset. ReLU activation function is used in the hidden nodes, sigmoid activation at output node and bias at all nodes are 0. Assume derivative of ReLU(x) at x=0 is 1, if needed.

- i) If one-step untargeted Fast Gradient Sign Method is used, calculate the adversarial input sample for instance #4. $\epsilon = 0.25$.

$$\mathbf{X}^{adv} = \mathbf{X} + \epsilon \text{sign}(\nabla_{\mathbf{X}} J(\mathbf{X}, y_{true}))$$

Data Instance #	Input x1	Input x2	True y	Left hidden output h1	Right hidden output	Actual y
4	1	1	1	3	1.5	0.817574

True y = 1, BCE becomes , BCE = - [1 log (y') + (1-1) log (1- y')] = - log y'

$$\frac{dL}{dx_1} = -(1 - O) \cdot \sum_{j=1}^m w_{h_j, o} \cdot w_{x_1, h_j} \cdot 1(h_j > 0)$$

dLoss/dx1 dLoss/dx2 x1_adv x2_adv
0 -0.27364 1 0.75

Loss= -ln(actual y)

dLoss/dx1=-1/actual y * actual y * (1-actual y)*(1-1)=0

dLoss/dx2=-1/actual y * actual y * (1-actual y)*(2-0.5)

- ii) If a two-step (iterative) untargeted Fast Gradient Sign Method is used, calculate the adversarial input sample for instance #4. Clipping parameter $\epsilon = 0.1$. $\alpha=0.025$.

Data Instance #	Input x1	Input x2	True y	h1	h2	y
4	1	1	1	3	1.5	0.817574

x1_adv x2_adv x1_adv x2_adv
bfore bfore after after
dLoss/dx1 dLoss/dx2 clipping clipping clipping clipping
0 -0.27364 1 0.975 1 0.975

2nd iteration with modified input

Data Instance #	Input x1	Input x2	True y	h1	h2	y
4	1	0.975	1	2.95	1.4875	0.811915

x1_adv x2_adv x1_adv
bfore bfore after
dLoss/dx1 dLoss/dx2 clipping clipping clipping
0 -0.28213 1 0.95 1

- iii) If one-step targeted Fast Gradient Sign Method is used, calculate the adversarial input sample for instance #4. Data Instance #4 is targeted, i.e., target y for instance #1 is set to 0. Rest of the true values are unchanged. $\epsilon = 0.25$.

$$\mathbf{X}^{adv} = \mathbf{X} - \epsilon \text{sign}(\nabla_{\mathbf{X}} J(\mathbf{X}, y_{target}))$$

Data Instance #	Input x1	Input x2	Target y
4	1	1	0

h1 h2 y
3 1.5 0.817574

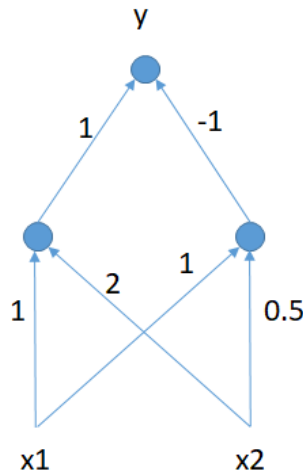
For $y = 0$, BCE = $-\log(1 - y')$

$$\frac{dL}{dx_1} = 0 \cdot \sum_{j=1}^m w_{h_j, 0} \cdot w_{x_1, h_j} \cdot 1(h_j > 0)$$

dLoss/dx1 dLoss/dx2 x1_adv x2_adv
0 1.226362 1 0.75

Loss = $-\ln(1 - \text{actual } y)$

Data Instance #	Input x1	Input x2	True y
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1



Q.3. Consider the following training dataset relating features x_1 (100's of sq. ft. of a flat) and x_2 (distance of the flat from the city centre) and its price y . $Y = a \cdot x_1 + b \cdot x_2^{0.5}$ is used as the regression model. No additional scaling is used. **[5+8=13 Marks]**

Instance #	x_1 (100's of sq. ft.)	x_2 (distance in km)	Y (price in lakhs of rupees)
1	5	1	25
2	7.5	4	35
3	8	9	35
4	12	16	50

- A. Find the optimal values of a and b and calculate the best price estimate for a flat of 1000 sq ft. (i.e., $x_1=10$) and at a distance of 12 km.

x_1 (100's of sq. ft.)	x_2 (distance in km)	Y (price in lakhs of rupees)	$\sqrt{x_2}$
5	1	25	1
7.5	4	35	2
8	9	35	3
12	16	50	4

$$\beta = (X^T X)^{-1} X^T Y$$

$$X = \begin{bmatrix} 1 & 5 & 1.0 \\ 1 & 7.5 & 2.0 \\ 1 & 8 & 3.0 \\ 1 & 12 & 4.0 \end{bmatrix}$$

$$Y = \begin{bmatrix} 25 \\ 35 \\ 35 \\ 50 \end{bmatrix}$$

Optimal regression coefficients:

$$\begin{matrix} a & b \\ 5.7377 & -4.2623 \end{matrix}$$

- B. Obtain the LIME explanation for the flat 1000 sq. ft. (i.e., $x_1=10$) and at a distance of 12 km. Use the following perturbations: (10, 12.5), (10, 11), (11.5, 12) and (9, 12). Use a Gaussian weighting function with variance 10.

Perturbed x_1	Perturbed x_2	Original predicted price	Gaussian weight
10	12	42.61196	
10	12.5	42.30749	0.987578
10	11	43.24055	0.951229
11.5	12	51.21851	0.893597
9	12	36.87426	0.951229

$$\text{Weight}(x) = \exp\left(-\frac{(d^2)}{2\sigma^2}\right)$$

- d is the Euclidean distance from the point of interest (10, 12)
- σ^2 is the variance (in this case, 10).

Gaussian weighted Linear model $y_{rel} = P \cdot x_{1_rel} + Q \cdot x_{2_rel}$ is fit around the point of interest (10,12)

$$\beta = (X^T W X)^{-1} X^T W Y$$

Relative x_1 w.r.t. (10,12)	Relative x_2 w.r.t. (10,12)	Related predicted price w.r.t (10,12)
--	--	---

$$X = \begin{bmatrix} 1 & 10 & 12 \\ 1 & 10 & 12.5 \\ 1 & 10 & 11 \\ 1 & 11.5 & 12 \\ 1 & 9 & 12 \end{bmatrix} \quad Y = \begin{bmatrix} 42.61196 \\ 42.30749 \\ 43.24055 \\ 51.21851 \\ 36.87426 \end{bmatrix}$$

$$\begin{matrix} x_{1_rel} = x_1^{perturbed} - x_1^{original} = x_1^{perturbed} - 10 \\ x_{2_rel} = x_2^{perturbed} - x_2^{original} = x_2^{perturbed} - 12 \end{matrix} \quad \begin{matrix} 0 & 0 & 0 \\ 0 & 0.5 & -0.30447 \\ 0 & -1 & 0.62859 \\ 1.5 & 0 & 8.60655 \\ -1 & 0 & -5.7377 \end{matrix} \quad W = \begin{bmatrix} 0.951229 \\ 0.987578 \\ 0.951229 \\ 0.893597 \\ 0.951229 \end{bmatrix}$$

Optimal $P = 5.7401193$ and $Q = -0.6283020$